MATH 5718: Applied Linear Algebra, Fall 2025

Course Project

Due: Wednesday, December 3, 11:59PM

Learning Outcomes

By completing this project, students will:

- Understand QR decomposition and its implementation.
- Understand the difference between full and reduced QR decomposition for non-square matrices.
- Solve least squares problems using QR decomposition.
- Apply linear algebra techniques to real-world regression problems.
- Implement iterative algorithms for eigenvalue computation.

Submission Instructions

- Use **Google Colab** or a similar environment to run your Python code.
- Submit a single PDF report that includes:
  1. Your code (formatted and readable).
  2. Explanations of how your code works.
  3. Results and requested comparisons (e.g., norms, shapes, eigenvalues).
  4. Discussion of outcomes and any observations.
- Include screenshots or outputs from Colab where relevant.
- Ensure your report is well-organized and clearly labeled for each question.
- You are welcome to modify or build upon any code snippet included in this document.

Question 1: Compute the QR decomposition of the matrix:

$$A = \begin{bmatrix} 1 & 3 & 3 \\ 2 & 2 & -2 \\ -2 & 2 & 1 \end{bmatrix}$$

using the Gram-Schmidt process implemented with two nested `for` loops.

```
import numpy as np

A = np.array([[1, 3, 3],
              [2, 2, -2],
              [-2, 2, 1]], dtype=float)

m, n = A.shape
Q = np.zeros((m, n))
R = np.zeros((n, n))

for j in range(n):
    v = A[:, j]
    for i in range(j):
        R[i, j] = np.dot(Q[:, i], A[:, j])
        v = v - R[i, j] * Q[:, i]
    R[j, j] = np.linalg.norm(v)
    Q[:, j] = v / R[j, j]
```

(a) Explain how this code works (step-by-step).

(b) Use `numpy.linalg.qr` to compute Q_{numpy} and R_{numpy}.

(c) Compare your computed Q with Q_{numpy} and your computed R with R_{numpy} using a matrix norm of your choice.

(d) You may notice a difference between your Q, R and NumPy's result. Explain why QR decomposition is not unique and implement a fix.

Question 2: Compute the QR decomposition of the following matrix:

$$A = \begin{bmatrix} -1 & -3 & 1 \\ -2 & 2 & -2 \\ 2 & -2 & 0 \\ 4 & 0 & -2 \end{bmatrix}$$

where $A \in \mathbb{R}^{4\times 3}$ (more rows than columns). Use `numpy.linalg.qr` in both modes (do not use the code provided before for implementing QR):

- `mode='reduced'`
- `mode='complete'`

(a) Compute the QR decomposition in both modes and display the shapes of Q and R.

(b) Explain the difference between the two modes and why the dimensions of Q and R change.

(c) Verify that $A = QR$ holds for both cases.

Question 3: Consider the overdetermined system of linear equations

$$Ax = y,$$

where $A \in \mathbb{R}^{m\times n}$ with $m > n$ (a tall matrix). We aim to find the least squares solution

$$x^* = \arg\min_x \|Ax - y\|_2^2.$$

Using the QR decomposition. If $A = QR$ with $Q \in \mathbb{R}^{m\times n}$ having orthonormal columns and $R \in \mathbb{R}^{n\times n}$ upper triangular (from the `mode='reduced'` decomposition), then:

$$Ax = y \quad \Longrightarrow \quad QRx = y.$$

Multiplying both sides by $Q^\top$ (since $Q^\top Q = I$):

$$Rx = Q^\top y.$$

Because R is upper triangular and square, we can solve for x by *back-substitution*.

Back-substitution algorithm. If $R \in \mathbb{R}^{n\times n}$ is upper triangular and $b \in \mathbb{R}^n$, the back-substitution to solve $Rx = b$ is

$$\text{for } i = n-1, n-2, \ldots, 0: \qquad x_i = \frac{1}{R_{ii}}\Big(b_i - \sum_{j=i+1}^{n-1} R_{ij}\, x_j\Big).$$

```
import numpy as np

def back_substitution(R, b):
    n = R.shape[0]
    x = np.zeros(n, dtype=float)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - np.dot(R[i, i+1:], x[i+1:])) / R[i, i]
    return x
```

Let:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \\ 3 & 2 \\ 4 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}.$$

(a) Explain how the back-substitution code works.

(b) Implement the QR-based least squares solution for the overdetermined system $Ax = y$.

(c) Compare your solution with the result from `numpy.linalg.lstsq` (using a suitable norm).

Question 4: Linear regression can be formulated as a least squares problem. Given a dataset with features $X \in \mathbb{R}^{m \times n}$ and target values $y \in \mathbb{R}^m$, the goal is to find a weight vector $w \in \mathbb{R}^n$ that minimizes:

$$\min_w \|Xw - y\|_2^2.$$

This is equivalent to solving the overdetermined system:

$$Xw \approx y,$$

where X is the design matrix (each row is a data point, each column is a feature).

(a) Load the `California Housing` dataset from `sklearn.datasets`. This dataset provides features such as median income and average rooms per household, with the target being the median house value (in hundreds of thousands of dollars). In this case, $m = 20{,}640$ and $n = 8$.

(b) Standardize the features.

(c) Formulate the linear regression problem as a least squares problem:

$$w^* = \arg\min_w \|Xw - y\|_2^2.$$

Solve for w^* using QR decomposition with back-substitution (as in the previous problem).

```
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
import numpy as np

# Load California Housing dataset
```

```
data = fetch_california_housing()
X = data.data  # Features
y = data.target  # Target

# Standardize features
scaler = StandardScaler()
X_design = scaler.fit_transform(X)
```

Question 5: The QR algorithm is a fundamental numerical method for computing eigenvalues of a matrix. Instead of solving the characteristic polynomial directly (which is computationally expensive and numerically unstable for large matrices), the QR algorithm uses iterative similarity transformations to approximate eigenvalues.

Key Idea. For a square matrix $A \in \mathbb{R}^{n\times n}$:

- Compute the QR decomposition:

$$A_k = Q_k R_k,$$

  where Q_k is orthogonal and R_k is upper triangular.

- Form the next iterate:

$$A_{k+1} = R_k Q_k.$$

  This is a similarity transformation, so A_{k+1} has the same eigenvalues as A_k.

- Repeat until A_k converges to an upper triangular matrix (Schur form). The diagonal entries approximate the eigenvalues.

Consider the matrix:

$$A = \begin{bmatrix} 4 & 1 & -2 \\ 1 & 2 & 0 \\ -2 & 0 & 3 \end{bmatrix}.$$

(a) Using the QR algorithm discussed, compute the eigenvalues of A.

(b) Use `numpy.linalg.eig` to directly compute the eigenvalues of A.

(c) Compare the two sets of eigenvalues and comment on the accuracy and convergence.